FACILITATOR QUESTION BANK

Day 3 — Anticipated Questions and Model Answers

Doubt clarification for RAG, tokenisation, MCP and observability

Questions participants are likely to raise, with answers written the way you would say them in the room. Difficult and challenging questions are marked, with guidance on handling them without losing the room or overclaiming.

Enterprise AI Engineering Bootcamp · Companion to the Day 3 presentation and study material

A. Tokenisation and Context

Q Modern models have huge context windows. Why not just pass the whole manual?

Three reasons, and the first is the one that surprises people. Accuracy degrades as irrelevant context grows — information buried in the middle of a large context is recalled substantially less reliably than information at the start or end. Second, latency rises roughly with input length. Third, cost rises directly. Passing twenty passages instead of five frequently answers worse while costing three times as much. Measure it on your own data in the lab; nobody believes it until they see it.

Q What chunk size should we use?

Chunk size matters less than chunk strategy. A 512-token section-aware chunk that respects headings and keeps procedures intact will beat a 512-token fixed chunk that cuts a safety procedure in half. If you want a starting point: 400 to 800 tokens with 10 to 15% overlap, section-aware. Then measure with your own questions, because the right answer depends on how your documents are structured.

Q Is overlap not just wasteful duplication?

It duplicates content and it buys boundary coverage. Without overlap, a passage split across a chunk boundary becomes effectively unretrievable — neither chunk contains enough of it to match. Ten to fifteen per cent is a reasonable insurance premium. If you have genuinely reliable section boundaries, you can reduce it, because the structure is doing the job overlap was compensating for.

Q Why are our costs so much higher than we estimated?

Almost always retrieved context. Teams estimate from the length of the user's question and the answer, which together are a few hundred tokens, and forget that eight retrieved passages are six to twelve thousand. The second common cause is tool schemas — every tool you attach is charged on every request whether it is used or not. Twenty tools attached to every call is a permanent tax on the whole system.

Q Does prompt compression help?

Sometimes, and it is rarely the first thing to reach for. Reducing the number of passages by reranking gives you a larger cost reduction and improves accuracy at the same time; compression reduces cost and risks losing the detail that mattered. Rerank first, cut the passage count, and only consider compression if you still have a budget problem afterwards.

B. Ingestion and Document Handling

Q Our PDF extraction seems to work. Why invest in layout-aware parsing?

Check the tables. Naive extraction usually produces readable prose and destroys tables, which is where torque values, tolerances, part compatibility and clearances live in technical documentation. The chunk looks fine in the database; the row-to-value relationship is gone. That failure is silent — you will not see an error, you will see a wrong torque figure with a confident citation. Ask one question that requires reading a table and see what comes back.

Q Some of our manuals are scanned images. Is OCR good enough?

Modern OCR handles clean scans well and struggles with poor ones — faded print, skewed pages, handwriting, dense engineering drawings. The important part is not the OCR quality itself but that you measure it. Sample fifty pages, check

them, and record the error rate per document type. Then set expectations accordingly, and consider excluding document classes where OCR quality is too poor rather than silently indexing unreliable text.

Q How do we handle diagrams and exploded views?

Three options in increasing order of cost. Extract the caption and surrounding text and index those, accepting that the diagram itself is not searchable. Use a vision model to generate a description at ingestion and index that alongside. Or use a multimodal retrieval approach that indexes the image directly. Most industrial teams get a long way with the first two, and the second is a good balance — the description is generated once and reviewed once, rather than at query time.

Q Our documents are updated constantly. How do we keep the index current?

Event-driven ingestion for anything safety-relevant and batch for the rest. When a document is published or superseded, that event triggers re-indexing of the affected content — not a nightly batch, because a system confidently citing a withdrawn procedure between the publication and the batch run is a real risk. For general reference material, nightly is fine and considerably simpler to operate.

Q Should we delete superseded documents?

No. Mark them superseded and record what replaced them. You need them for three reasons: audit, equipment still in the field running the older revision, and questions about what the guidance used to be. Filter to current content by default and allow an explicit override for historical queries. Deletion loses information you cannot recover.

C. Retrieval

Q Our retrieval finds similar documents but not the right one. What is wrong?

Most likely one of three things, in order of probability. Chunking has separated the answer from the context that makes it findable. Dense-only retrieval is failing on the identifiers in the query — part numbers, error codes — which is what hybrid search fixes. Or there is no reranking, so a marginally relevant passage that happens to embed closely is outranking the correct one. Run the lab comparison on your own corpus; it will tell you which.

Q Do we really need both dense and sparse retrieval?

For technical documentation, almost certainly yes. Your queries mix natural language and exact identifiers in the same sentence: "what is the torque spec for the mounting bolts on HX-4471-B". Dense handles the first half and can quietly fail on the second, returning a plausible passage about a different part. Sparse handles the second and fails on the first. Hybrid is typically the single largest retrieval improvement available on industrial corpora.

Q What exactly does a reranker do that retrieval does not?

Retrieval embeds the query and the document separately, so it can never model how they interact — that limitation is precisely why it is fast enough to search millions of documents. A reranker processes the query and each candidate together, which models the interaction directly and is far more accurate, and far too slow to run over the whole corpus. Retrieve fifty with the fast method, rerank them with the accurate one, pass the top four. You get the recall of the first and the precision of the second.

Q Should we use a knowledge graph?

Eventually, perhaps. Not first. Graph RAG is genuinely powerful for questions about how things relate — which components share a failure mode, what else this fault affects — and it is expensive to build and to maintain, because someone has to keep the entity relationships correct. On industrial documentation, hybrid retrieval plus reranking plus metadata filtering usually gets further, faster. Ask what specific question you cannot answer without it, and if you can name one, that is the business case.

Q How do we stop the system answering when it should not know?

Two parts. Set a threshold on the reranker score below which no passage is considered adequate, and instruct the model explicitly to answer only from the provided passages and to say when they are insufficient. Then measure it: write ten questions your documentation genuinely does not cover and see how often the system correctly declines. Most teams doing this for the first time find their system answers all ten confidently.

Q Would fine-tuning on our documents remove the need for retrieval?

No, and it would make things worse in a specific way. Fine-tuning teaches the model the style and vocabulary of your documents, not their content reliably. You would get fluent, confident, wrong answers about your equipment — and with no citation to check, an engineer has no way to detect them. Retrieval gives you facts you can trace to a source. Those are different mechanisms solving different problems.

D. MCP and Integration

Q Why use MCP rather than just calling our APIs directly?

For a single application calling a single stable API in a fixed pipeline, direct is fine and simpler. MCP earns its place when several AI applications need the same capability, when the capability is owned by another team, or when the model

decides at runtime whether to use it. The real argument is organisational: eight governed servers consumed by every application, rather than forty bespoke integrations of varying quality, each with its own security review.

Q Is MCP a standard we can rely on, or will it change?

It is being actively developed and adopted, and specific details do change. Two things protect you. First, design your servers so the tool interface alone is sufficient — support for other primitives varies by host application. Second, keep the server as a thin protocol layer over an internal interface you control, so a protocol change means updating the wrapper rather than the capability. That is ordinary defensive design and it applies to any evolving standard.

Q How do we stop the model calling a tool it should not?

Not through the prompt. Four controls outside the model: an explicit allow-list determining which roles may invoke which tools; server-side authorisation that checks the caller independently; approval gates on actions above a cost or reversibility threshold; and validation at the tool itself, because the arguments were generated by a model that may have read content an attacker controlled. The model must never be the thing deciding what it is permitted to do.

Q What is the confused deputy problem?

A privileged component acting on behalf of an unprivileged caller without checking what that caller is allowed to do. It is very easily introduced in MCP architectures, because the natural implementation is for the server to hold one high-privilege credential to the underlying system. A user who may see nothing asks a question, the server queries with full privilege, and the answer contains everything. Identity must flow end to end and the server must make its own authorisation decision.

Q Should we wrap our retrieval in MCP too?

Only if the model decides whether retrieval happens. If retrieval always runs before generation, it is a pipeline stage, not a tool, and wrapping it adds latency and indirection for nothing. If you have several retrieval sources and the model chooses between them, or another application needs the same retrieval capability, then it is a genuine tool and MCP fits.

E. Evaluation and Observability

Q How do we evaluate a RAG system when answers are free text?

Split it. Retrieval is evaluated against known correct passages, which is objective and cheap — context recall and precision. Generation is evaluated for faithfulness to the passages, which can be done by a model with human spot-checking, and for citation accuracy, which can be validated programmatically. That separation is the whole trick: an end-to-end score tells you the system is wrong without telling you which half to fix.

Q Can we use a model to grade the answers?

For faithfulness and relevance, yes, with two conditions. Validate the grader against human judgement on a sample first — measure the agreement rate, and if it is poor, the grader is not usable. And never use the same model that generated the answer to grade it, because it will systematically approve of its own reasoning. For citation accuracy, do not use a model at all: check programmatically that the cited passage was retrieved and contains the claim.

Q Which metric should we report to management?

Three, not one. Answer correctness on the gold set, because that is what they asked about. Citation accuracy, because that is what determines whether engineers actually use it. And no-answer accuracy, because a system that never declines is a system that will eventually be confidently wrong in a way someone remembers. Add cost per query if there is a budget conversation, which there usually is.

Q Our metrics look fine but users complain. Why?

Three usual causes. Your evaluation questions are easier than real ones — build the gold set from actual support tickets and search logs, not from what you thought people would ask. Your metrics are aggregates hiding a bad slice — break them down by product family, document type, and question type. Or the complaints are about latency, formatting or trust rather than correctness, which no accuracy metric captures. Look at the feedback text, not just the score.

Q What is the minimum observability we need?

One thing above all: record exactly what was passed to the model, with token counts — the context-assembly span. When a user reports a wrong answer, that single record usually answers the question immediately, and almost no proof of concept captures it. After that: retrieval scores, tool call outcomes, latency by stage, and cost per request. That is perhaps a day of work and it converts debugging from guesswork into reading.

Q How often should we run the regression suite?

On every prompt, model, embedding, chunking or retrieval change, and on a weekly schedule regardless of changes. The scheduled run is the one that catches drift, which by definition happens without anyone changing anything. If the suite is slow, split it: a fast subset on every change and the full suite weekly.

F. Difficult Questions — Handle With Care

> **These challenge the premise or put you on the spot. Answer honestly rather than defensively.**

Q This is a lot of engineering. Could we not just buy a RAG product?

You can, and for a general document assistant it may well be the right call. What a product will not do for you is the parts that are specific to your documents and your organisation: table-aware ingestion of your manuals, your metadata schema, your access model, your evaluation set, and your version and supersession handling. Those are the things that determine

whether it works on industrial documentation — and they are exactly what today covered. Buy the platform if you like; you still have to do this work.

Q Our RAG system already works well. Is any of this necessary?

How do you know it works well? That is not a rhetorical question — it is the same challenge as Day 1. If the evidence is that the demos go well and nobody has complained loudly, that is limited evidence. Build the gold set with known correct passages, run it, and look at the retrieval metrics in particular. If your context recall is genuinely high, you will have proved it and you can move on to cost and latency. That is a good outcome.

Q We tried RAG and the answers were poor, so we moved on. Was it the wrong approach?

Possibly, and more likely it was a retrieval problem diagnosed as a model problem. The most common version of this: fixed-size chunking that destroyed table structure, dense-only retrieval that could not match part numbers, no reranking, and twenty passages passed to the model. Every one of those is fixable in days, and together they routinely take a system from unusable to useful. Before abandoning the approach, run the five-strategy comparison on your corpus.

Q You keep saying to measure. We do not have the people for that.

The minimum is smaller than it sounds. Fifty questions with the correct passage recorded, taken from real support tickets, is two or three days of one person's time — and it is the instrument you use for the next two years. Without it you are making changes and hoping. With it, you can tell in an afternoon whether reranking helped. It is not overhead on the work; it is what makes the work compound.

Q Different teams here are building very different things. Is Day 3 relevant to all of us?

Directly relevant to the knowledge assistant and the research assistant. Partly relevant to the agentic use case, which retrieves as one of its steps. Less directly relevant to the vision and maintenance work, though the evaluation discipline, the observability design and the MCP module apply to all five. If your system does not retrieve, treat Modules 4 and 5 as the core of your day and use Modules 1 to 3 to understand what your colleagues are building.

> **General principle.** Where the honest answer is "your problem may be simpler than this", say so. The credibility gained makes the parts you do insist on — measurement, citation validation, no-answer detection — considerably easier to argue for.

G. Quick Reference — One-Line Answers

If asked	Say
Why not pass the whole manual?	Accuracy degrades with irrelevant context, and cost and latency rise.
What chunk size?	Strategy matters more than size. Section-aware, 400–800 tokens, 10–15% overlap.
Why are costs so high?	Retrieved context, and tool schemas charged on every request.
PDF extraction seems fine?	Check the tables. That is where it silently fails.
Delete superseded documents?	Never. Mark superseded, filter by default, allow override.
Dense or sparse?	Both. Your queries mix prose and part numbers in one sentence.
What does reranking add?	It models query-document interaction, which retrieval structurally cannot.
Knowledge graph?	Name the question you cannot answer without it. If you can, build it.
Fine-tune instead of retrieve?	No. You would get fluent wrong answers with no citation to check.

MCP or direct API?	Does the model decide whether it happens? If not, it is not a tool.
Stop unsafe tool calls?	Allow-lists, server-side authorisation, approval gates, argument validation.
How to evaluate free text?	Split it. Retrieval against known passages; generation for faithfulness.
Model as grader?	Validate it against humans first, and never the model that wrote the answer.
Minimum observability?	Record exactly what was passed to the model, with token counts.
Metrics fine, users unhappy?	Your questions are easier than real ones, or an aggregate is hiding a slice.

Companion documents: **Day 3 Presentation** (44 slides with speaker notes) and **Day 3 Study Material** (concept explanations with analogies and technical depth).